

COMSATS UNIVERSITY ISLAMABAD, ATTOCK CAMPUS

Department of Computer Sciences



Course Name: Information Security Lab

Assignment # 04

Submitted by: Muhammad Usama

Registration No: FA24-BSE-062

Instructor: Mam Ambreen Gul

Date: 08 May,2026

Task 1: Simple RSA Implementation (Without Libraries)

Code:

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Function to calculate modular inverse
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d
    return None

# Function for RSA Encryption
def encrypt(message, e, n):
    encrypted = []

    for char in message:
        # Convert character into ASCII value
        m = ord(char)

        # RSA Encryption Formula
        c = (m ** e) % n

        encrypted.append(c)

    return encrypted

# Function for RSA Decryption
def decrypt(ciphertext, d, n):
    decrypted = ""

    for c in ciphertext:
        # RSA Decryption Formula
        m = (c ** d) % n

        # Convert ASCII back to character
        decrypted += chr(m)

    return decrypted
```

```

# ----- RSA Key Generation -----

p = 17
q = 11

# Step 2: Calculate n
n = p * q

# Step 3: Calculate Euler Totient Function
phi = (p - 1) * (q - 1)

# Step 4: Choose e
e = 7

# Check if e and phi are coprime
if gcd(e, phi) == 1:
    print("e is valid")
else:
    print("Choose another e")

# Step 5: Calculate d
d = mod_inverse(e, phi)

# Display Keys
print("Public Key (e, n): ", (e, n))
print("Private Key (d, n): ", (d, n))

# Message Encryption

message = "USAMA"

print("\nOriginal Message:", message)

encrypted_message = encrypt(message, e, n)

print("Encrypted Message:", encrypted_message)

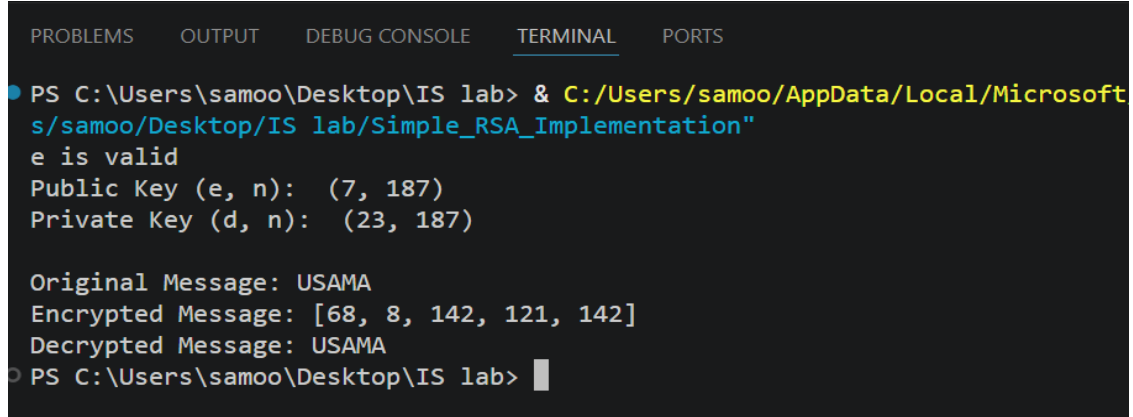
# ----- Message Decryption -----

decrypted_message = decrypt(encrypted_message, d, n)

print("Decrypted Message:", decrypted_message)

```

Output:

A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected), and 'PORTS'. The terminal shows a command prompt where a file named 'Simple_RSA_Implementation' is executed. The output displays the RSA parameters: 'e is valid', 'Public Key (e, n): (7, 187)', and 'Private Key (d, n): (23, 187)'. It then shows the encryption and decryption of the message 'USAMA', resulting in the encrypted list '[68, 8, 142, 121, 142]' and the decrypted message 'USAMA'.

```
PS C:\Users\samoo\Desktop\IS lab> & C:/Users/samoo/AppData/Local/Microsoft
s/samoo/Desktop/IS lab/Simple_RSA_Implementation"
e is valid
Public Key (e, n): (7, 187)
Private Key (d, n): (23, 187)

Original Message: USAMA
Encrypted Message: [68, 8, 142, 121, 142]
Decrypted Message: USAMA
PS C:\Users\samoo\Desktop\IS lab> █
```

Line By Line Explanation:

`gcd()` Function

```
def gcd(a, b):
```

This function is created to calculate the Greatest Common Divisor.

```
while b != 0:
```

Loop runs until b becomes 0.

```
a, b = b, a % b
```

Euclidean Algorithm is used.

```
return a
```

Returns final GCD value.

`mod_inverse()` Function

```
def mod_inverse(e, phi):
```

This function calculates private key d.

```
for d in range(1, phi):
```

Checks every value from 1 to phi.

```
if (d * e) % phi == 1:
```

Condition for modular inverse.

return d

Returns the correct value of d.

Prime Numbers

p = 11

q = 13

Two prime numbers are selected.

Calculate n

$n = p * q$

Multiplies both prime numbers.

Result:

143

Calculate phi

$\phi = (p - 1) * (q - 1)$

Calculates Euler Totient Function.

Result:

120

Public Key e

e = 7

Public exponent is selected.

Verify e

if $\gcd(e, \phi) \neq 1$:

Checks whether e and phi are coprime.

Private Key d

$d = \text{mod_inverse}(e, \phi)$

Calculates modular inverse.

Result:

103

Message

```
message = "USAMA"
```

Text message that will be encrypted.

ASCII Conversion

```
message_numbers = [ord(char) for char in message]
```

Converts characters into ASCII values.

Example:

U = 85

S = 83

Encryption

```
cipher = (num ** e) % n
```

RSA encryption formula is applied.

Formula:

$$C = M^e \bmod n$$

Decryption

```
plain = (cipher ** d) % n
```

RSA decryption formula is applied.

Formula:

$$M = C^d \bmod n$$

Convert Back to String

```
final_message = ''.join(decrypted)
```

Combines characters into original message.

Advantages of RSA

1. RSA provides secure communication.
2. Public and private keys are different.
3. Private key is never shared.
4. RSA is widely used in HTTPS and secure systems.

Task 2 – Digital Signature Using RSA

Code:

```
# Task 2: Digital Signature Using RSA

import hashlib
# Function to calculate GCD

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Function to calculate modular inverse

def mod_inverse(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d

# RSA Key Generation
p = 17
q = 11

n = p * q
phi = (p - 1) * (q - 1)

e = 7

if gcd(e, phi) != 1:
    print("Invalid e")
```

```

d = mod_inverse(e, phi)

# Public and Private Keys
public_key = (e, n)
private_key = (d, n)

print("Public Key:", public_key)
print("Private Key:", private_key)

# Original Message
message = "Information Security Lab"

# Step 1: Create SHA256 Hash
hash_value = hashlib.sha256(message.encode()).hexdigest()

print("Original Hash:", hash_value)

# Convert hash into integer
hash_int = int(hash_value, 16)

# Step 2: Create Digital Signature
# Signature = hash^d mod n

signature = pow(hash_int, d, n)

print("Digital Signature:", signature)

```

Output:

```

PS C:\Users\samoo\Desktop\IS lab> & C:/Users/samoo/AppData/Local/Microsoft/WindowsApps/python3.10.9/python.exe C:/Users/samoo/Desktop/IS lab/Digital_Signature.py
Public Key: (7, 187)
Private Key: (23, 187)
Original Hash: 490572601061b11440ac11cff7a7529b4cd153837b66f09216346b4e9b403a69
Digital Signature: 70
PS C:\Users\samoo\Desktop\IS lab>

```

Line by Line Explanation:

Import hashlib

import hashlib

This library is used to create SHA256 hash.

RSA Key Generation

$p = 17$

$q = 11$

Two prime numbers are selected.

$n = p * q$

Calculates n.

Result:

187

$\phi = (p - 1) * (q - 1)$

Calculates Euler Totient Function.

Result:

160

$e = 7$

Public exponent.

$d = \text{mod_inverse}(e, \phi)$

Private key is calculated.

Result:

23

Message Hashing

`hash_value = hashlib.sha256(message.encode()).hexdigest()`

Creates SHA256 hash of message.

Hash functions convert data into fixed-size output.

Convert Hash to Integer

```
hash_int = int(hash_value, 16)
```

Converts hexadecimal hash into integer.

Signature Creation

```
signature = pow(hash_int, d, n)
```

Creates digital signature using private key.

Formula:

$$\text{Signature} = \text{Hash}^d \bmod n$$

Signature Verification

```
verified_hash = pow(signature, e, n)
```

Decrypts signature using public key.

Formula:

$$\text{Hash} = \text{Signature}^e \bmod n$$

Compare Hashes

```
if verified_hash == original_hash_check:
```

Checks whether both hashes match.

If matched:

Message is authentic.

Modified Message Test

```
modified_message = "Information Security lab"
```

Message is slightly modified.

Only one letter is changed.

Verification Failure

if verified_hash == (modified_hash_int % n):

Hashes become different.

Verification fails.

This proves message integrity.

Advantages of Digital Signature

1. Confirms sender identity.
2. Prevents message tampering.
3. Provides data integrity.
4. Prevents denial by sender.

Why Verification Fails After Modification

Even a small change in message changes SHA256 hash completely.

Example:

Original:

Information Security Lab

Modified:

Information Security lab

Only one letter changes, but hash becomes completely different.

This property is called:

Avalanche Effect

Real World Applications

Digital signatures are used in:

- Banking systems
- Software verification
- E-commerce
- Secure emails
- Blockchain technology